Table of Contents

1.1. A Bit of History
1.2. The Java Virtual Machine
1.3. Organization of the Specification
1.4. Notation
1.5. Preview Features
1.5.1. Restrictions on the Use of Preview Features
1.5.2. Current Preview VM Features
1.6. Feedback

Chapter 1. Introduction

1.1. A Bit of History

The Java® programming language is a general-purpose, concurrent, object-oriented language. Its syntax is similar to C and C++, but it omits many of the features that make C and C++ complex, confusing, and unsafe. The Java platform was initially developed to address the problems of building software for networked consumer devices. It was designed to support multiple host architectures and to allow secure delivery of software components. To meet these requirements, compiled code had to survive transport across networks, operate on any client, and assure the client that it was safe to run.

The popularization of the World Wide Web made these attributes much more interesting. Web browsers enabled millions of people to surf the Net and access media-rich content in simple ways. At last there was a medium where what you saw and heard was essentially the same regardless of the machine you were using and whether it was connected to a fast network or a slow modem.

Web enthusiasts soon discovered that the content supported by the Web's HTML document format was too limited. HTML extensions, such as forms, only highlighted those limitations, while making it clear that no browser could include all the features users wanted. Extensibility was the answer.

The HotJava browser first showcased the interesting properties of the Java programming language and platform by making it possible to embed programs inside HTML pages. Programs are transparently downloaded into the browser along with the HTML pages in which they appear. Before being accepted by the browser, programs are carefully checked to make sure they are safe. Like HTML pages, compiled programs are network- and host-independent. The programs behave the same way regardless of where they come from or what kind of machine they are being loaded into and run on.

A Web browser incorporating the Java platform is no longer limited to a predetermined set of capabilities. Visitors to Web pages incorporating dynamic content can be assured that their machines cannot be damaged by that content. Programmers can write a program once, and it will run on any machine supplying a Java run-time environment.

1.2. The Java Virtual Machine

The Java Virtual Machine is the cornerstone of the Java platform. It is the component of the technology responsible for its hardware- and operating system-independence, the small size of its compiled code, and its ability to protect users from malicious programs.

The Java Virtual Machine is an abstract computing machine. Like a real computing machine, it has an instruction set and manipulates various memory areas at run time. It is reasonably common to implement a programming language using a virtual machine; the best-known virtual machine may be the P-Code machine of UCSD Pascal.

The first prototype implementation of the Java Virtual Machine, done at Sun Microsystems, Inc., emulated the Java Virtual Machine instruction set in software hosted by a handheld device that resembled a contemporary Personal Digital Assistant (PDA). Oracle's current implementations emulate the Java Virtual Machine on mobile, desktop and server devices, but the Java Virtual Machine does not assume any particular implementation technology, host hardware, or host operating system. It is not inherently interpreted, but can just as well be implemented by compiling its instruction set to that of a silicon CPU. It may also be implemented in microcode or directly in silicon.

The Java Virtual Machine knows nothing of the Java programming language, only of a particular binary format, the `class` file format. A `class` file contains Java Virtual Machine instructions (or *bytecodes*) and a symbol table, as well as other ancillary information.

For the sake of security, the Java Virtual Machine imposes strong syntactic and structural constraints on the code in a `class` file. However, any language with functionality that can be expressed in terms of a valid `class` file can be hosted by the Java Virtual Machine. Attracted by a generally available, machine-independent platform, implementors of other languages can turn to the Java Virtual Machine as a delivery vehicle for their languages.

The `class` file format is versioned: every `class` file declares a version number, of the form *major.minor*, which indicates the file's dependency on a particular release of Java SE, and influences the interpretation of the file by the Java Virtual Machine.

The Java Virtual Machine specified here is compatible with Java SE 26, and supports the Java programming language specified in *The Java Language Specification, Java SE 26 Edition*. It supports `class` files with major version numbers 45 through 70, inclusive.

Tools that generate `class` files will typically adopt the latest major version number in order to take advantage of the latest features; but a `class` file with an older major version number can generally expect to be supported in future Java Virtual Machine releases.

For reference, the following table shows the `class` file major version numbers supported by each release of Java SE, up to Java SE 26. The third column, "Earliest", shows the earliest `class` file major version number supported by the Java Virtual Machine in that release. The fourth column, "Latest", shows the latest `class` file major version number supported by the Java Virtual Machine in that release. (For very early releases, the JDK version is shown instead of the Java SE release.)

Table 1.2-A. Java SE releases & `class` file major versions

Java SE	Released	Earliest	Latest
1.0.2	May 1996	45	45
1.1	February 1997	45	45
1.2	December 1998	45	46
1.3	May 2000	45	47
1.4	February 2002	45	48
5.0	September 2004	45	49
6	December 2006	45	50
7	July 2011	45	51
8	March 2014	45	52

Java SE	Released	Earliest	Latest
9	September 2017	45	53
10	March 2018	45	54
11	September 2018	45	55
12	March 2019	45	56
13	September 2019	45	57
14	March 2020	45	58
15	September 2020	45	59
16	March 2021	45	60
17	September 2021	45	61
18	March 2022	45	62
19	September 2022	45	63
20	March 2023	45	64
21	September 2023	45	65
22	March 2024	45	66
23	September 2024	45	67
24	March 2025	45	68
25	September 2025	45	69
26	March 2026	45	70

1.3. Organization of the Specification

Chapter 2 gives an overview of the Java Virtual Machine architecture.

Chapter 3 introduces compilation of code written in the Java programming language into the instruction set of the Java Virtual Machine.

Chapter 4 specifies the `class` file format, the hardware- and operating system-independent binary format used to represent compiled classes and interfaces.

Chapter 5 specifies the start-up of the Java Virtual Machine and the loading, linking, and initialization of classes and interfaces.

Chapter 6 specifies the instruction set of the Java Virtual Machine, presenting the instructions in alphabetical order of opcode mnemonics.

Chapter 7 gives a table of Java Virtual Machine opcode mnemonics indexed by opcode value.

In the Second Edition of *The Java® Virtual Machine Specification*, Chapter 2 gave an overview of the Java programming language that was intended to support the specification of the Java Virtual Machine but was not itself a part of the specification. In *The Java Virtual Machine Specification, Java SE 26 Edition*, the reader is referred to *The Java Language Specification, Java SE 26 Edition* for information about the Java programming language.

In the Second Edition of *The Java® Virtual Machine Specification*, Chapter 8 detailed the low-level actions that explained the interaction of Java Virtual Machine threads with a shared main memory. In *The Java Virtual Machine Specification, Java SE 26 Edition*, the reader is referred to Chapter 17 of *The Java Language Specification, Java SE 26 Edition* for information about threads and locks. Chapter 17 reflects *The Java Memory Model and Thread Specification* produced by the JSR 133 Expert Group.

1.4. Notation

Throughout this specification, we refer to classes and interfaces drawn from the Java SE Platform API. Whenever we refer to a class or interface (other than those declared in an example) using a single identifier `N`, the intended reference is to the class or interface named `N` in the package `java.lang`. We use the fully qualified name for classes or interfaces from packages other than `java.lang`.

Whenever we refer to a class or interface that is declared in the package `java` or any of its subpackages, the intended reference is to that class or interface as loaded by the bootstrap class loader (§5.3.1).

Whenever we refer to a subpackage of a package named `java`, the intended reference is to that subpackage as determined by the bootstrap class loader.

A cross-reference within this specification is shown as (§x.y). Occasionally, we refer to concepts in the *The Java Language Specification, Java SE 26 Edition* via cross-references of the form (JLS §x.y).

The use of fonts in this specification is as follows:

- A `fixed width` font is used for Java Virtual Machine data types, exceptions, errors, `class` file structures, Prolog code, and Java code fragments.
- *Italic* is used for Java Virtual Machine "assembly language", its opcodes and operands, as well as items in the Java Virtual Machine's run-time data areas. It is also used to introduce new terms and simply for emphasis.

Non-normative text, designed to clarify the normative text of this specification, is given in smaller, indented text.

This is non-normative text. It provides intuition, rationale, advice, examples, etc.

1.5. Preview Features

A *preview feature* is:

- a new feature of the Java programming language ("preview language feature"), or
- a new feature of the Java Virtual Machine ("preview VM feature"), or

- a new module, package, class, interface, field, method, constructor, or enum constant in the `java.*` or `javax.*` namespace ("preview API")

that is fully specified, fully implemented, and yet impermanent. It is available in implementations of a given release of the Java SE Platform to provoke developer feedback based on real world use; this may lead to it becoming permanent in a future release of the Java SE Platform.

The preview features defined by a given release of the Java SE Platform are enumerated in the Java SE Platform Specification for that release. The preview features are specified as follows:

- Preview language features are specified in standalone documents that indicate changes ("diffs") to *The Java® Language Specification* for that release. The specifications of preview language features are incorporated into *The Java® Language Specification* by reference, and made a part thereof, if and only if preview features are enabled at compile time.
- Preview VM features are specified in standalone documents that indicate changes ("diffs") to *The Java® Virtual Machine Specification* for that release. The specifications of preview VM features are incorporated into *The Java® Virtual Machine Specification* by reference, and made a part thereof, if and only if preview features are enabled at run time.
- Preview APIs are specified within the Java SE API Specification for that release.

1.5.1. Restrictions on the Use of Preview Features

Implementations of the Java SE Platform disable, at both compile time and run time, the preview features defined by a given release, unless the user indicates via the host system, at both compile time and run time, that preview features are enabled. Implementations do not provide a way to enable only some of the given release's preview features.

A `class` file *depends on the preview features of Java SE* `N` (`N` ≥ 12) if:

- the `class` file's major version number is the latest major version number supported by Java SE `N` in Table 1.2-A, and
- the `class` file's minor version number is 65535

For example, a `class` file with version number 66.65535 depends on the preview features of Java SE 22, because 66 is the latest major version number supported by Java SE 22.

Compilers may need to emit `class` files that depend on the preview features of a given release even if the `class` files do not use any preview VM features. For example, a Java source file that uses a preview language feature of the release must be compiled to a `class` file that depends on the preview features of the release.

At run time, the rules for loading a `class` file that depends on the preview features of a given Java SE Platform release are specified in §4.1. Such a `class` file is tied to that release of the Java SE Platform, and cannot be loaded on any other release (even if preview features are enabled) because the preview features it depends on may be different or missing in the other release.

1.5.2. Current Preview VM Features

Java SE 26 does not define any preview VM features.

1.6. Feedback

Readers are invited to report technical errors and ambiguities in *The Java® Virtual Machine Specification* to `jls-jvms-spec-comments@openjdk.org`.

Questions concerning the generation and manipulation of `class` files by `javac` (the reference compiler for the Java programming language) may be sent to `compiler-dev@openjdk.org`.

Legal Notice